```
trait Prop {
  def check: Either[(FailedCase, SuccessCount), SuccessCount]
}
```

In the case of failure, `check` returns a `Left((s,n))`, where `s` is some `String` that represents the value that caused the property to fail, and `n` is the number of cases that succeeded before the failure occurred.

That takes care of the return value of `check`, at least for now, but what about the arguments to `check`? Right now, the `check` method takes no arguments. Is this sufficient? We can think about what information `Prop` will have access to just by inspecting the way `Prop` values are created. In particular, let's look at `forAll`:

```
def forAll[A](a: Gen[A])(f: A => Boolean): Prop
```

Without knowing more about the representation of `Gen`, it's hard to say whether there's enough information here to be able to generate values of type `A` (which is what we need to implement `check`). So for now let's turn our attention to `Gen`, to get a better idea of what it means and what its dependencies might be.

8.2.3 The meaning and API of generators

We determined earlier that a `Gen[A]` was something that knows how to generate values of type `A`. What are some ways it could do that? Well, it could *randomly* generate these values. Look back at the example from chapter 6—there, we gave an interface for a purely functional random number generator `RNG` and showed how to make it convenient to combine computations that made use of it. We could just make `Gen` a type that wraps a `State` transition over a random number generator:[4]

```
case class Gen[A](sample: State[RNG,A])
```

EXERCISE 8.4

Implement `Gen.choose` using this representation of `Gen`. It should generate integers in the range `start` to `stopExclusive`. Feel free to use functions you've already written.

```
def choose(start: Int, stopExclusive: Int): Gen[Int]
```

EXERCISE 8.5

Let's see what else we can implement using this representation of `Gen`. Try implementing `unit`, `boolean`, and `listOfN`.

```
def unit[A](a: => A): Gen[A]
```

← Always generates the value `a`

[4] Recall the definition: `case class State[S,A](run: S => (A,S))`.